

**STREDNÁ PRIEMYSELNÁ ŠKOLA ELEKTROTECHNICKÁ
ZOCHOVA 9, BRATISLAVA**



Autonómny dron s počítačovým videním

projekt z praktickej časti odbornej zložky maturitnej skúšky

Meno kandidáta: Erik Štvrtecký

Trieda: 4.D

Vedúci práce: Mgr. Jakub Krcho

Šk. rok: 2024/2025

**STREDNÁ PRIEMYSELNÁ ŠKOLA ELEKTROTECHNICKÁ
ZOCHOVA 9, BRATISLAVA**

Autonómny dron s počítačovým videním

Meno kandidáta: Erik Štvrtecký

Trieda: 4.D

Vedúci práce: Mgr. Jakub Krcho

Rozsah: 38 strán, 9 obrázkov

Odovzdané:

Podpis vedúceho práce:

STREDNÁ PRIEMYSELNÁ ŠKOLA ELEKTROTECHNICKÁ ZOCHOVA 9, 811 03 BRATISLAVA



Študijný odbor: 2573M00 - programovanie digitálnych technológií
Meno: Erik Štvrtecký
Trieda: IV.D
Školský rok: 2024/2025

Zadanie praktickej časti odbornej zložky maturitnej skúšky formou obhajoby vlastného projektu

Názov práce: Autonómny dron s počítačovým videním

Úloha:

Navrhnete program pre dron, ktorý bude pomocou počítačového videnia sledovať a pohybovať sa podľa čiernej čiary pod ním. Dron by mal počas svojej cesty rozpoznať aj rôzne farebné útvary, ktoré nadväzujú na definované úlohy. Tieto úlohy musí dron splniť. Využite dron aj na prepravu balíka, ktorý dron musí bezpečne prepraviť na koniec svojej trasy. Po vykonaní hlavnej úlohy dron pristane na určenom mieste.

Technické parametre:

- programovanie v jazyku Python
- programovanie vo VS Code
- použitie knižnice OpenCV
- 3D tlač

Písomná časť:

Odovzdať vo viazanej forme (hrebeňová, termoväzba, lepená...) formátu A4 s počtom strán min. 20, bude obsahovať:

- zadanie praktickej maturitnej práce - originál
- čestné prehlásenie
- poďakovanie (nie je povinné)
- zoznam obrázkov, tabuliek
- obsah
- úvod
- popis použitých technológií (max. na 5 strán) - teoretická časť
- popis technického riešenie - praktická časť
- použitá literatúra podľa normy ISO 690
- záver
- prílohy

Stredná priemyselná škola
elektrotechnická
811 03 Bratislava, Zochova 9



Ing. Marián Beniák, PhD.
riaditeľ

V Bratislave, dňa 31.03.2025

Čestné prehlásenie

Čestne prehlasujem, že problematiku týkajúcu sa náplne zadaného projektu v rámci praktickej časti odbornej zložky maturitnej skúšky formou obhajoby som spracoval sám za pomoci uvedenej použitej literatúry. Inú, ako uvedenú literatúru som nepoužil.

V Bratislave dňa:

Erik Štvrtecký

Pod'akovanie

Chcel by som sa pod'akovať svojmu vedúcemu práce nielen za pomoc a vedenie pri tvorbe maturitného projektu, ale aj za celé štyri roky, ktorý vo mňa veril a naučil ma mnoho do budúcnosti.

OBSAH

Úvod	1
1 Automatizácia a autonómne systémy	2
2 Drony a ich technológie	3
3 Používané technológie	5
3.1 Dron DJI Tello	5
3.2 Programovací jazyk Python	6
3.3 Knižnica DJITelloPy	7
3.4 Knižnice OpenCV a NumPy	7
3.5 Knižnica Tkinter	8
3.6 Qt framework a knižnica PySide6	8
3.7 Modelovanie v programe Blender a 3D tlač	9
4 Prvé kroky praktickej realizácie	10
4.1 Návrh a koncept softvéru	10
4.2 Konštrukcia a tlač súčiastok	11
4.2.1 Držiak na zrkadlo	13
4.2.2 Konštrukcia na simulovanú prepravu balíka	14
4.3 Vývojový diagram autonómneho letu	15
5 Počítačové videnie	17
5.1 Detekcia farieb	18
5.1.1 Inicializácia triedy	19
5.1.2 Rozpoznávanie jednotlivých farieb	19
5.1.3 Odstránenie šumu v maske	20
5.2 Rozpoznávanie útvarov	20
5.2.1 Inicializácia triedy	21
5.2.2 Detekcia útvarov	21
5.2.3 Identifikácia útvarov	22
5.2.4 Sledovanie a stabilizácia detegovaných útvarov	22
5.2.5 Aktualizácia sledovaných útvarov	23
6 Pohyb drona	23
6.1 Sledovanie čiary	24
6.2 Riadenie pohybu drona	25
6.3 Plnenie úloh podľa farebných útvarov	25
7 Grafické rozhranie programu	26
Záver	28

ZOZNAM OBRÁZKOV

Obrázok 1 - Auto značky Tesla, ktoré ponúka autonómne riadenie.....	3
Obrázok 2 - Svetelná šou s dronmi.....	5
Obrázok 3 - Dron DJI Tello.....	6
Obrázok 4 - Analyzovanie drona.....	12
Obrázok 5 - Prvý návrh držiaka na zrkadlo.....	13
Obrázok 6 - Finálna verzia držiaka na zrkadlo.....	14
Obrázok 7 - Zacvakávací mechanizmus.....	14
Obrázok 8 - Finálny dizajn konštrukcie na simulovanú prepravu balíka.....	15
Obrázok 9 - Vývojový diagram autonómneho letu.....	17

ZOZNAM SKRATIEK

Skratka	Popis
API	Rozhranie pre programovanie aplikácií (Application Programming Interface)
GCS	Pozemná riadiaca stanica (Ground Control Station)
GUI	Grafické používateľské rozhranie (Graphical User Interface)
IoT	Internet vecí (Internet of Things)
QML	Qt modelový jazyk (Qt Modeling Language)
SDK	Softvérový vývojársky balík (Software Development Kit)
UI	Používateľské rozhranie (User Interface)
VR	Virtuálna realita (Virtual Reality)

Úvod

Technológia autonómnych dronov sa v posledných rokoch výrazne rozvíja a nachádza uplatnenie v mnohých odvetviach, vrátane logistiky, monitoringu, poľnohospodárstva a záchranných operácií. Schopnosť dronu pohybovať sa samostatne bez priamej intervencie človeka otvára nové možnosti pre automatizáciu úloh a zlepšovanie efektivity rôznych procesov.

V rámci maturitnej práce sa zameriavam na vývoj autonómneho dronu, ktorý pomocou počítačového videnia dokáže sledovať čiernu čiaru na zemi, rozpoznávať a reagovať na rôzne farebné útvary, a vykonať definované úlohy vrátane simulovanej prepravy balíka, ktorá nenaruší stabilitu letu.

Cieľom mojej maturitnej práce je navrhnúť a implementovať program pre dron, ktorý bude schopný splniť tieto komplexné úlohy a následne bezpečne pristáť na určenom mieste. Realizácia tohto projektu si vyžaduje kombináciu rôznych technológií a znalostí, vrátane programovania v jazyku Python, použitia knižnice OpenCV na spracovanie obrazu, použitie knižnice PySide6 na vytvorenie používateľského rozhrania a 3D tlače na výrobu potrebných súčiastok.

Motiváciou pre túto tému je preukázanie schopnosti spojiť moderné technológie s praktickým riešením reálnych problémov. S narastajúcim dopytom po autonómnych systémoch je dôležité pochopiť, ako fungujú a ako ich možno implementovať tak, aby vykonávali presné úlohy, či už na pomoc v priemysle, alebo v iných oblastiach spoločnosti.

TEORETICKÁ ČASŤ

V teoretickej časti sa zameriame na základy autonómnych systémov a technológie, ktoré sú kľúčové pri dronoch. Rozoberieme si teoretické základy použitých technológií v maturitnej práci, ako dron DJI Tello, programovací jazyk Python a potrebné knižnice, ako OpenCV, NumPy, DJITelloPy, Tkinter a PySide6, ktoré tvoria základ nášho softvéru. Okrem toho sa budeme venovať aj technológii 3D tlače a modelovaniu v programe Blender.

1 Automatizácia a autonómne systémy

V dnešnej dobe zohráva automatizácia spolu s autonómnymi systémami kľúčovú úlohu v rôznych oblastiach nášho každodenného života. Od inteligentných domácich spotrebičov až po priemyselné roboty vo fabrikách a autonómne vozidlá, nám tieto technológie menia spôsob, akým pracujeme, cestujeme a komunikujeme so svetom okolo nás. Hoci sa pojmy automatizácia a autonómne systémy často zamieňajú, existuje medzi nimi jasný rozdiel.

Automatizácia je aplikácia technológií, programov, robotiky alebo procesov na dosiahnutie výsledkov s minimálnym zásahom človeka. Je čoraz bežnejšia v modernom svete a má nespočetné množstvo aplikácií, vrátane podnikových, priemyselných a spotrebiteľských aplikácií. Automatizačné softvéry a technológie sú využívané v rôznych odvetviach, ako napríklad financie, zdravotníctvo, energetika či obrana krajiny. Organizácie ju využívajú na zvýšenie produktivity a zisku, na zlepšenie zákazníckych služieb a ich spokojnosti, na zníženie nákladov a prevádzkových chýb, a na väčšiu efektivitu prevádzky. Automatizácia prináša rôzne výhody a v konečnom dôsledku môže viesť k lepším výsledkom pre organizácie a ich zákazníkov [1].

Na druhej strane, autonómne systémy, ktoré sú základom umelej inteligencie, idú ešte ďalej. Sú schopné vykonávať úlohy bez priameho zásahu človeka a dosiahnuť rovnaké výsledky bez ohľadu na prostredie, v ktorom sa nachádzajú [2]. Medzi príklady tejto technológie patria autonómne vozidlá a drony alebo chatboti. Tieto systémy sú navrhnuté tak, aby vedeli fungovať samostatne a rozhodovať sa na základe algoritmov, podľa ktorých sú navrhnuté a na základe dát, ktoré neustále zbierajú

zo svojho okolia. Zbieranie a analýza dát im umožňuje neustále sa zlepšovať, lepšie sa prispôbovať dynamickým podmienkam a reagovať na zmeny v prostredí. Taktiež tieto systémy sa musia riadiť zásadou, že ich činnosť nesmie ohroziť človeka, majetok alebo samotný systém. Táto prispôsobivosť im umožňuje adaptovať sa na rôzne podmienky a vykonávať úlohy, ktoré by inak vyžadovali priamy zásah človekom [3].

Automatizácia a autonómne systémy predstavujú významné kroky vpred v rozvoji moderných technológií, ktoré menia náš život a priemysel. Aj keď sa tieto technológie vyvíjajú rýchlym tempom, prinášajú nielen výhody, ale aj nové výzvy. Vyžadujú si neustály výskum a vývoj predovšetkým v oblastiach ako etika, bezpečnosť a regulácie. V budúcnosti sa očakáva, že tieto technológie budú zohrávať zásadnú úlohu v ďalšej zmene nášho každodenného života a pracovného prostredia, tak ako je tomu aj dnes.



Obrázok 1 - Auto značky Tesla, ktoré ponúka autonómne riadenie

2 Drony a ich technológie

Dron, známy aj ako bezpilotné lietadlo, je lietajúce zariadenie, ktoré môže byť riadené autonómne alebo na diaľku, bez potreby pilota na palube. Drony sa vyrábajú v rôznych veľkostiach, ale najčastejšie ide o malé alebo stredne veľké modely. Využívajú sa v rôznych oblastiach, od tajných vojenských operácií až po donášku zásielok alebo snímanie okolia.

Ako funguje dron a aké sú jeho súčiastky? Dron funguje vďaka kombinácii hardvéru a softvéru, ktoré zabezpečujú jeho stabilitu a ovládateľnosť. Zvyčajne je vybavený rotormi, vrtuľami alebo pevnými krídlami, senzormi a kamerami, navigačným systémom (GPS) a gyroskopmi na udržanie stability. Drony sú najčastejšie

napájané lítium-polymérovými (Li-Po) batériami, ktoré poskytujú dostatočnú energiu na ich prevádzku. Okrem batérií existujú aj alternatívne formy pohonu, ako vodíkové palivové články či spaľovacie motory, ktoré sa používajú u väčších dronov s dlhšou dobou letu. Medzi jeho najdôležitejšie komponenty patrí aj pozemná riadiaca stanica (GCS), ktorá umožňuje operátorom komunikovať s dronom zo zeme. Niektoré modely sú schopné prepravovať užitočné zaťaženie, ako napríklad medicínske či komerčné zásielky. Na prenos údajov medzi dronom a operátorom slúžia dátové spojenia, napríklad Wi-Fi alebo satelitné pripojenie.

Existujú rôzne typy dronov, z ktorých je každý navrhnutý pre špecifické podmienky a požiadavky odvetví. Kým niektorí potrebujú ľahký a kompaktný dron na držanie kamery, iní vyžadujú väčšie modely na prepravu nákladu. Viacrotorové drony sú populárne v komerčnom a rekreačnom sektore vďaka svojej jednoduchosti a dostupnosti. Naopak, jednorotorové drony, ktoré sa podobajú helikoptéram, sú využívané na prepravu ťažších nákladov a vykonávanie geodetických prác. Drony s pevnými krídlami sú ideálne pre dlhšie lety a zložité úlohy, ako napríklad vojenské operácie. Existujú aj iné typy dronov, ktoré sú špeciálne navrhnuté pre konkrétne účely, napríklad záchranné drony na vykonávanie misií v nebezpečných podmienkach.

Drony sa stali kľúčovým nástrojom v rôznych odvetviach vďaka ich všestranosti a neustálemu technologickému pokroku. S pomocou umelej inteligencie a strojovému učeniu sa ich autonómne schopnosti výrazne zlepšujú, čo im umožňuje vykonávať náročnejšie úlohy. Vďaka integrácii s technológiami internetu vecí (IoT) sú schopné v reálnom čase zdieľať údaje s inými dronmi a operátormi, čím sa zvyšuje ich koordinácia a efektivita. V budúcnosti môžeme očakávať rozvoj tzv. „roja dronov“ (drone swarms), kde viaceré drony budú koordinovať a synchronizovať svoje pohyby a spolupracovať na rôznych úlohách. Tieto roje môžu byť využívané pri svetelných šou ako alternatíva ohňostrojev alebo pri bezpečnostných a záchranných operáciách. Avšak s týmto rozvojom prichádzajú aj nové výzvy, ako je ochrana súkromia a etické používanie dronov. S rastúcou komplexnosťou a potenciálnym vojenským využitím je nevyhnutné prijať prísne regulácie, ktoré zabezpečia zodpovedné využívanie tejto technológie a predišlo sa ich zneužitiu [4].



Obrázok 2 - Svetelná šou s dronmi

3 Použité technológie

V projekte sme použili rôzne technológie, ktoré nám umožnili dosiahnuť požadované výsledky. Kombinácia hardvéru a softvéru, od programovacieho jazyka Python cez frameworky na tvorbu používateľských rozhraní (UI) až po špecializovaný program na tvorbu 3D modelov, zohrala kľúčovú úlohu pri implementácii celkového riešenia.

3.1 Dron DJI Tello

DJI Tello je kompaktný a programovateľný dron vyvinutý spoločnosťou Ryze Tech v spolupráci s DJI a Intelom. Je vhodný najmä pre začiatočníkov a študentov, ktorí sa chcú oboznámiť s dronmi a ich programovaním. Vďaka jednoduchému ovládaniu je ideálny na výučbu a experimentovanie v oblasti autonómneho lietania. Na rozdiel od iných dronov, ktoré sa dajú ovládať len prostredníctvom klasických ovládačov, ho vieme ovládať aj pomocou mobilnej aplikácie Tello App a je kompatibilný aj s okuliarmi na virtuálnu realitu (VR), čo umožňuje pohlčujúcejší zážitok z letu [5].

Dron DJI Tello je možné programovať vo vzdelávacej mobilnej aplikácii Tello Edu, ktorá podporuje vizuálny blokový programovací jazyk Scratch. Pomocou neho môžu používatelia vytvárať jednoduché programy na ovládanie drona a odosielať mu príkazy, ktoré následne vykoná. Vďaka svojej jednoduchosti je Scratch obľúbený pri výučbe programovania cez vytváranie vlastných interaktívnych príbehov, hier a animácií. Používatelia si pri práci so Scratchom rozvíjajú logické a kritické myslenie, učia sa riešiť problémy a spolupracovať s inými [6].

Okrem blokového programovania v Scratchi sa dá dron ovládať aj pomocou programovacieho jazyka Python, čo prináša viac možností pre pokročilejších používateľov. Na tento účel slúži Tello Software Development Kit (SDK) verzie 2.0, ktorý poskytuje rozhranie pre odosielanie príkazov na ovládanie a prijímanie dát z drona. DJI Tello má zabudovaný Wi-Fi modul, ktorý umožňuje vytváranie vlastnej siete. Tello SDK následne využíva toto pripojenie a zabezpečuje nám tak efektívnu komunikáciu s dronom. Na prenos príkazov sa používa protokol UDP, ktorý zohráva kľúčovú úlohu pri efektívnej komunikácii s dronom a prenose dát [7].

Prečo práve protokol UDP? Oproti protokolu TCP je výhodnejší v situáciách, kde rýchlosť je dôležitejšia než celková spoľahlivosť prenosu. UDP nevyžaduje potvrdenie doručenia paketov v správnom poradí a bez chýb, čím sa minimalizuje oneskorenie, a umožňuje tak rýchlu výmenu dát. Je to ideálne pre aplikácie, ktoré pracujú s dátami v reálnom čase, ako je napríklad aj ovládanie dronov a kde menšie straty neovplyvnia celkovú funkčnosť systému [8].



Obrázok 3 - Dron DJI Tello

3.2 Programovací jazyk Python

Python je jeden z najpopulárnejších programovacích jazykov, ktorý sa vyznačuje jednoduchou a čitateľnou syntaxou. Ide o interpretovaný, objektovo orientovaný, vysokoúrovňový programovací jazyk, ktorý je vďaka svojej flexibilita a jednoduchosti obľúbený medzi programátormi a vývojármi. Vďaka dynamickému písaniu a vstavaným dátovým štruktúram je Python ideálny na rýchly vývoj aplikácií. Jeho jednoduchá syntax, ktorá sa dá ľahko naučiť, dbá na čitateľnosť a znižuje náklady na údržbu kódu. Podporuje rozsiahlu knižnicu modulov a balíkov, čo umožňuje modularitu programu a opätovné použitie kódu [9].

V porovnaní s inými programovacími jazykmi je Python známy svojou jednoduchosťou a vysokou úrovňou abstrakcie, čo uľahčuje prácu s komplexnými konceptmi. Na rozdiel od C alebo C++, ktoré vyžadujú manuálnu správu pamäte, Python ju spravuje automaticky. Oproti Jave nevyžaduje rozsiahle deklarácie typov pri práci s dátovými štruktúrami, čo urýchľuje vývoj. Zatiaľ čo jazyky ako JavaScript sú primárne určené na vývoj webových stránok, je Python univerzálnejší a používa sa v širokej škále oblastí od dátovej analýzy až po umelú inteligenciu [10].

Python je využívaný aj v oblastiach, ako napríklad počítačové videnie, strojové učenie a robotika. Vďaka jeho flexibilitě a rozsiahlej komunite vývojárov je Python voľbou pre rýchly vývoj a implementáciu nových funkcií v systémoch pre autonómne zariadenia, vrátane autonómnych dronov.

3.3 Knižnica DJITelloPy

Na programovanie drona DJI Tello v Pythone sa využíva knižnica DJITelloPy, ktorá slúži ako rozhranie pre programovanie aplikácií (API) pre jednoduché ovládanie drona. Táto knižnica poskytuje spôsob, ako komunikovať s dronom prostredníctvom Wi-Fi a pracuje aj s oficiálnym Tello SDK. Umožňuje nám plný prístup k funkciám drona, ako je pohyb, získavanie telemetrických údajov alebo obrazový výstup z kamery. Okrem toho nám ponúka aj možnosť implementácie pokročilých funkcií, ako je plánovanie trasy, počítačové videnie alebo autonómne riadenie drona [11].

3.4 Knižnice OpenCV a NumPy

OpenCV (Open Source Computer Vision Library) je multiplatformová open-source knižnica určená na spracovanie obrazu a počítačové videnie, ktorá ponúka širokú škálu nástrojov na analýzu vizuálnych dát. Je veľmi populárna v oblasti strojového učenia, robotiky, bezpečnosti a autonómnych systémov. OpenCV poskytuje množstvo funkcií na detekciu objektov, sledovanie pohybu, rozpoznávanie tvárí, ako aj spracovanie videa a obrázkov v reálnom čase. Knižnica je napísaná v programovacom jazyku C++, ale je dostupná aj pre iné jazyky, ako je Python, čo ju robí flexibilnou pre rôzne aplikácie [12].

V kombinácii s OpenCV sa často používa knižnica NumPy, ktorá je jednou z najvýkonnejších knižníc na prácu s numerickými údajmi v jazyku Python. Poskytuje

nám rôzne nástroje a funkcie na efektívnu manipuláciu s veľkými maticami a viacrozmernými poľami. Vďaka tomu je nevyhnutnou súčasťou vedeckých výpočtov, spracovania dát a strojového učenia. Keďže aj obrazové dáta sú reprezentované ako polia pixelov, NumPy nám umožňuje rýchlu analýzu a transformáciu pomocou optimalizovaných matematických operácií [13].

3.5 Knižnica Tkinter

Tkinter je štandardná knižnica pre tvorbu grafických používateľských rozhraní (GUI) v Pythone. Je súčasťou štandardnej knižnice Pythonu, čo znamená, že je dostupná bez nutnosti inštalácie ďalších balíkov. Tkinter poskytuje nástroje na vytváranie okien, tlačidiel, textových polí a ďalších GUI komponentov, ktoré umožňujú jednoduchú a efektívnu interakciu medzi používateľom a aplikáciou. Je ideálnym riešením pre menšie aplikácie a prototypy, ktoré si vyžadujú rýchlu implementáciu vizuálneho rozhrania. Tkinter je postavený na knižnici Tcl/Tk a je známy svojou jednoduchosťou a flexibilitou, čo ho robí populárnym medzi začiatočníkmi aj skúsenými programátormi [14].

3.6 Qt framework a knižnica PySide6

PySide6 je oficiálna Python knižnica z projektu *Qt for Python*, ktorá poskytuje prístup ku Qt frameworku verzie 6.0 a vyššie, a umožňuje jeho prepojenie s Pythonom. Vďaka tejto knižnici vieme vytvárať aplikácie s natívnym alebo aj s vlastným používateľským rozhraním [15].

Čo je vlastne Qt framework? Qt je multiplatformový framework určený na vývoj počítačových aj mobilných aplikácií. Podporuje široké spektrum platforiem, medzi ktoré patria Linux, OS X, Windows, Android, iOS, BlackBerry a ďalšie.

Qt ponúka dve hlavné technológie, pomocou ktorých vieme vytvárať používateľské rozhrania. Sú to Qt Widgets a Qt Quick. Qt Widgets existuje už od vzniku Qt, čo z nej robí stabilnú a spoľahlivú technológiu. Umožňuje nám vytvárať klasické počítačové aplikácie s natívnym vzhľadom komponentov, avšak s obmedzenou možnosťou prispôsobenia vzhľadu. Qt Quick je modernejšia technológia, ktorá využíva Qt modelový jazyk (QML) na definovanie vlastného UI. Podporuje animácie,

responzívny dizajn a rôzne štýly, vďaka čomu je vhodná na tvorbu moderných aplikácií s dynamickým UI [16].

Pre tento projekt sme si zvolili technológiu Qt Quick, ktorá nám poskytuje väčšiu flexibilitu pri návrhu UI a umožňuje jednoduché vytváranie vizuálne atraktívnych aplikácií. Qt má zároveň veľkú a aktívnu komunitu vývojárov, čo zaručuje širokú podporu, množstvo dokumentácie a bohaté zdroje na riešenie problémov. Tento faktor bol kľúčový pri výbere tejto technológie, pretože zaručuje rýchlu implementáciu a prístup k osvedčeným riešeniam, ktoré výrazne zjednodušujú vývoj aplikácií.

3.7 Modelovanie v programe Blender a 3D tlač

3D tlač je technológia, ktorá umožňuje rýchly vývoj prototypov a presné vytváranie dielov na mieru. V oblasti vývoja komponentov, ako je aj konštrukcia na simulovanú prepravu balíka, poskytuje možnosť vytvoriť diely s presnými požiadavkami na tvar, pevnosť a hmotnosť. Táto technológia je ideálna pre vytváranie ľahkých, no na druhej strane pevných častí, ktoré musia byť špecificky prispôsobené úlohám, ako je manipulácia s predmetmi počas pohybu. 3D tlač umožňuje rýchle testovanie a úpravy dizajnu súčiastok alebo komponentov, čo výrazne urýchľuje proces vývoja a prispieva k efektívnej realizácii funkčných a bezpečných komponentov [17].

Na tvorbu 3D modelov určených na tlač sa často využívajú špecializované programy na návrh a modelovanie. Jedným z najpopulárnejších nástrojov v tejto oblasti je Blender. Je to open-source softvér určený na tvorbu a úpravu 3D objektov. Ponúka nám širokú škálu nástrojov na parametrické aj voľné modelovanie, prácu s textúrami, animácie a fyzikálne simulácie. Aj keď je primárne využívaný v oblasti grafiky, animácií a hier, vďaka jeho všestranosti je vhodný aj na vytváranie objektov určených na 3D tlač. Podporuje export modulov do formátov kompatibilných s 3D tlačiarňami, čo umožňuje jednoduchú prípravu návrhov na výrobu [18].

PRAKTICKÁ ČASŤ

V praktickej časti sa zameriame na realizáciu autonómneho drona, vrátane návrhu softvéru a hardvéru. Rozoberieme si prvé kroky pri vývoji, ako je návrh a koncept softvéru, konštrukcia a tlač súčiastok, a tvorba jednotlivých komponentov, ako sú držiak na zrkadlo a konštrukcia na simulovanú prepravu balíka. Ďalej sa budeme venovať vývoju autonómneho letu, počítačovému videniu a detekcii farieb a útvarov, ktoré sú kľúčové pre navigáciu a plnenie úloh dronom. Sledovanie čiary a pohyb drona v rámci daného prostredia budú základom pre vykonávanie konkrétnych a autonómnych úloh a rozhodnutí. Nakoniec sa zameriame na vytvorenie grafického rozhrania, ktoré umožní jednoduchú interakciu so systémom.

4 Prvé kroky praktickej realizácie

Realizácia tohto projektu si vyžadovala dôkladný postup pri návrhu, ako aj implementácií jednotlivých komponentov. Pred samotným programovaním bolo potrebné dôkladne analyzovať dostupné technológie, definovať kľúčové požiadavky systému a zvoliť správne riešenia, ktoré umožnia spoľahlivý autonómny let drona. V tejto kapitole si priblížime prvé kroky našej praktickej realizácie, od počiatočných úvah o hardvérovom a softvérovom riešení, cez návrh a výrobu doplnkových súčiastok, až po prispôsobenie sa technologickým obmedzeniam.

4.1 Návrh a koncept softvéru

Jedným z hlavných cieľov bolo vytvoriť softvérové riešenie, ktoré dokáže efektívne interpretovať vizuálne informácie a na základe nich zabezpečiť autonómne riadenie drona. Už v náčrtoch a počiatočných fázach vývoja bolo jasné, že softvér, ktorý naprogramujeme, musí byť navrhnutý modulárne, aby umožnil prehľadnú implementáciu jednotlivých častí systému. Tento prístup zároveň poskytuje flexibilitu pri neskorších úpravách a rozširovaní aplikácie. Rozhodli sme sa teda softvér rozdeliť do niekoľkých kľúčových modulov prepojených v hlavnej aplikácii, ktorá riadi celý proces a zabezpečuje komunikáciu medzi nimi.

Softvér sme teda rozdelili na počítačové videnie, riadenie drona a používateľské rozhranie. Počítačové videnie je zodpovedné za spracovanie obrazu z kamery drona, analýzu vizuálnych informácií a ich efektívne ukladanie na ďalšie použitie. Tento modul primárne využíva knižnicu OpenCV, ktorá výrazne uľahčuje spracovanie obrazu a detekciu objektov. Poskytuje nám rôzne nástroje, čo umožňuje rýchlu a presnú analýzu.

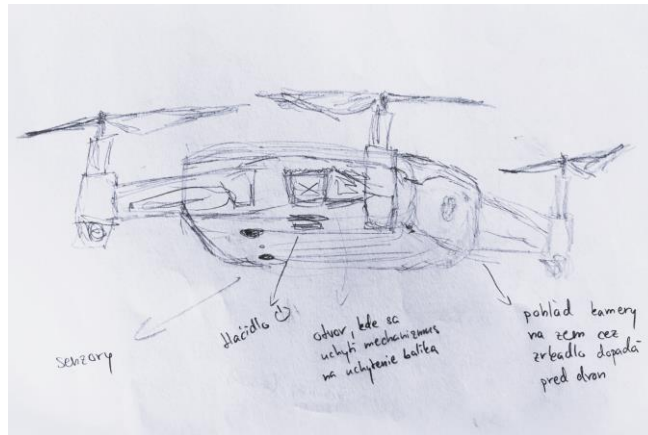
Modul na riadenie drona zabezpečuje autonómny let drona na základe spracovaných vizuálnych informácií. Využíva uložené dáta a následne vykonáva potrebné výpočty, ktoré slúžia na autonómne rozhodovanie v rôznych podmienkach. Na komunikáciu s hardvérom a získavanie telemetrických údajov sme využili knižnicu DJITelloPy, ktorá nám poskytuje aj prístup k ovládaniu drona. Tento modul je zodpovedný za to, aby dron vykonával správne kroky v súlade s analýzou, ktorú vykonáva počítačové videnie a reagoval na zmeny v prostredí.

Používateľské rozhranie slúži na interakciu medzi používateľom a softvérom. Umožňuje nám zobrazit' aktuálny stav drona, ako je jeho výška, teplota, stav batérie a ďalšie telemetrické údaje. Prostredníctvom tohto rozhrania môžeme monitorovať ďalšie kroky drona a jeho rozhodnutia v reálnom čase, čo poskytuje prehľad o jeho činnosti počas letu. Okrem iného nám umožňuje ovládanie drona, pripojenie sa k nemu a samotné spustenie autonómneho letu.

Tieto moduly spolupracujú tak, že aplikácia riadi celý proces a zabezpečuje komunikáciu medzi jednotlivými časťami. Počas autonómneho letu sa jednotlivé moduly navzájom dopĺňajú a koordinujú svoje činnosti. Celý softvér je navrhnutý tak, aby prebiehala plynulá výmena informácií, čo zaručuje efektívny a spoľahlivý autonómny let.

4.2 Konštrukcia a tlač súčiastok

Pri praktickej realizácii nášho projektu bolo potrebné navrhnuť a vytlačiť dodatočné komponenty, ktoré by zabezpečili správny chod autonómneho letu. Hlavnými 3D tlačenými súčiastkami boli držiak na zrkadlo a konštrukcia na simulovanú prepravu balíka.



Obrázok 4 - Analyzovanie drona

Držiak na zrkadlo bol navrhnutý tak, aby umožnil kamere dronu snímať obraz kolmo na zem, pričom zrkadlo bolo upevnené pod presne stanoveným uhlom. Táto konštrukcia zabezpečila, že modul počítačového videnia mal presný vizuálny vstup na snímanie a následnú analýzu terénu pod dronom. Konštrukcia na simulovanú prepravu balíka bola navrhnutá tak, aby umožnila testovanie autonómneho prenášania nákladu bez negatívneho vplyvu na stabilitu letu.

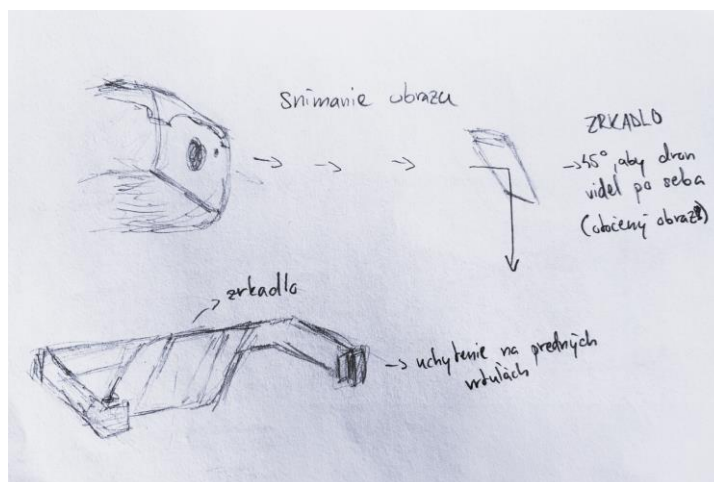
Na tlač súčiastok sme použili materiál PLA, známy aj ako polylaktid, ktorý sme zvolili kvôli jeho jednoduchosti spracovania, výhodnej cene a dostatočnej pevnosti pri nízkom zaťažení. Pevnosť materiálu bola kľúčová pre zaručenie stability a spoľahlivosti vytlačených dielov. Okrem toho je materiál relatívne ľahký, čo zavážilo pri výbere, nakoľko dron nemá dostatočný výkon na prepravu ťažších komponentov. PLA má oproti iným materiálom, ako ABS alebo PETG výhodu v nižšej potrebnej teplote a absencii nepríjemného zápachu počas tlače. Na druhej strane má nižšiu tepelnú odolnosť a je krehkejší, čo však v našom prípade nepredstavuje problém. Tento materiál je taktiež biorozložiteľný, čo prispieva k ekologickej udržateľnosti a znižuje negatívny dopad na životné prostredie.

Validácia vytlačených súčiastok prebiehala testovaním v praxi priamo na drone. Po pripevnení komponentov sa overovalo, či správne držia na svojom mieste, neovplyvňujú stabilitu drona počas letu a umožňujú bezproblémovú interakciu so softvérom.

4.2.1 Držiak na zrkadlo

Prvý dôležitý komponent, ktorý sme museli navrhnuť a vytlačiť, bol držiak na zrkadlo. Tento komponent zohráva podstatnú úlohu pri zabezpečení správneho fungovania počítačového videnia drona. Úloha držiaka je jednoduchá, no zároveň veľmi špecifická. Umožňuje kamere dronu snímať obraz kolmo na zem, kde sa nachádzajú dôležité vizuálne informácie pre navigáciu autonómneho letu. Taktiež zabezpečuje presný vizuálny vstup, ktorý je potrebný na analýzu terénu pod dronom.

Pôvodný návrh uchytenia bol umiestnený na predných vrtuliach, presnejšie v strede prednej časti konštrukcie drona, kde sa nachádzali otvory, ktoré by mohli slúžiť na uchytenie. Tento dizajn však bol problematický, pretože otvory boli veľmi široké a počas letu dochádzalo k uvoľňovaniu držiaka, čo spôsobovalo nestabilitu a občasné pády dielu. Preto sme sa rozhodli presunúť uchytenie na vrchnú časť predných vrtúl. Tento nový dizajn nám zaručil stabilnejšie uchytenie, pretože sa držiak už nevkladal do stredu, ale pevne sa nasadil na vrch, čo zaručilo jeho väčšiu stabilitu počas letu. Tento krok sa osvedčil, keďže nový spôsob uchytenia výrazne zlepšil držanie držiaka aj počas náročnejších letových podmienok.



Obrázok 5 - Prvý návrh držiaka na zrkadlo

Pri prenose obrazu sme zvažovali použiť zrkadlovú fóliu, keďže ponúkala jednoduchšiu aplikáciu a nízku hmotnosť. Po prvotných testoch sme však zistili, že výrazne skresľovala obraz a nedokázala dostatočne ostro odrážať detaily terénu. Rozhodli sme sa preto vyrezať si vlastné zrkadlo, ktoré poskytovalo kvalitnejší obraz bez výraznej deformácie. Zakúpili sme rezač na sklo a vyhotovili sme presný kus, ktorý bol následne osadený do držiaka. Zrkadlo sme následne umiestnili do držiaka

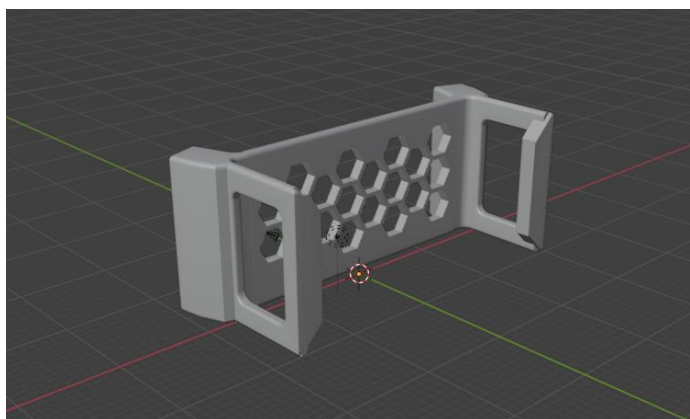
pod uhlom 45° . Pred kameru drona, ktorá je fixne namontovaná a štandardne sníma obraz priamo pred sebou, bolo nutné umiestniť zrkadlo na nasmerovanie pohľadu na zem. Toto zrkadlo pod správnym uhlom nám zabezpečilo to, že obraz bol presne premietnutý pred kameru drona. Týmto riešením sme dosiahli presný a spoľahlivý vizuálny vstup potrebný na navigáciu počas autonómneho letu.



Obrázok 6 - Finálna verzia držiaka na zrkadlo

4.2.2 Konštrukcia na simulovanú prepravu balíka

Konštrukcia na simulovanú prepravu balíka bola navrhnutá s dôrazom na pevné a bezpečné uchytenie nákladu bez negatívneho vplyvu na stabilitu letu. Pri návrhu bolo dôležité, aby balík nezasahoval do zorného poľa kamery ani neblokoval senzory na spodnej strane drona. Rovnako sme museli zohľadniť rozmery balíka tak, aby neprekážal pri vzlete a pristávaní. Celková hmotnosť musela byť čo najmenšia, aby dron nebol preťažený a mohol tak efektívne lietať.

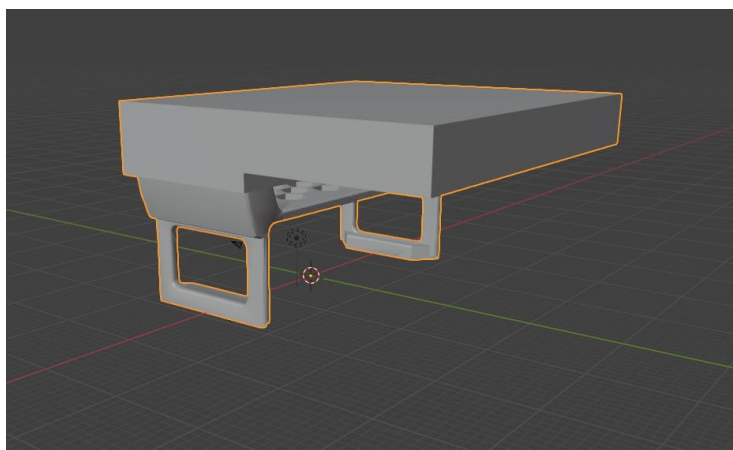


Obrázok 7 - Zacvakávací mechanizmus

Pri počiatkových náčrtoch sme sa najskôr zamerali na vytvorenie zacvakávacieho mechanizmu, ktorý sa uchyťí na bočných stranách drona. Tento

mechanizmus bol prioritne testovaný, aby sme sa uistili, že konštrukcia bude dostatočne pevná a odolná počas letu. Až po úspešnom testovaní sme pristúpili k návrhu samotného balíka, ktorého rozmery sme prispôbili mechanizmu a celkovej konštrukcii drona. Šírka balíka zodpovedá šírke tela drona, dĺžka je od senzorov po prednú časť, kde sa nachádza kamera a výška bola prispôbená tak, aby nepresahovala úroveň pristávacích nôh drona. Tieto presné parametre nám zabezpečia, aby dron vedel ľahko vzlietnuť, pristáť a snímať prostredie pomocou jeho senzorov a kamery. Vďaka použitému ľahkému materiálu sa nám podarilo dosiahnuť ideálnu rovnováhu medzi pevnosťou a hmotnosťou bez potreby kalibrácie softvéru drona.

Ďalšou výzvou, s ktorou sme sa stretli bolo zabezpečenie prístupu k dôležitým komponentom drona. Museli sme tak modifikovať uchytávací mechanizmus a vytvoriť otvory pre tlačidlo na vypnutie a zapnutie drona a na micro USB port, aby bolo možné dron pohodlne ovládať bez nutnosti demontáže konštrukcie. Okrem toho sme museli zohľadniť aj chladenie drona, ktoré sa nachádza na jeho spodnej strane. Balík preto nemohol priliehať priamo k telu drona, ale bolo nutné ponechať dostatočný priestor na cirkuláciu vzduchu, čím predídeme prehrievaniu. Všetky tieto faktory sme museli starostlivo zapracovať do dizajnu tak, aby bola konštrukcia praktická, funkčná a zároveň stabilná počas letu.



Obrázok 8 - Finálny dizajn konštrukcie na simulovanú prepravu balíka

4.3 Vývojový diagram autonómneho letu

Vývojový diagram autonómneho letu poskytuje vizuálny prehľad o jednotlivých krokoch, ktoré softvér vykonáva pri riadení drona. Diagram znázorňuje proces

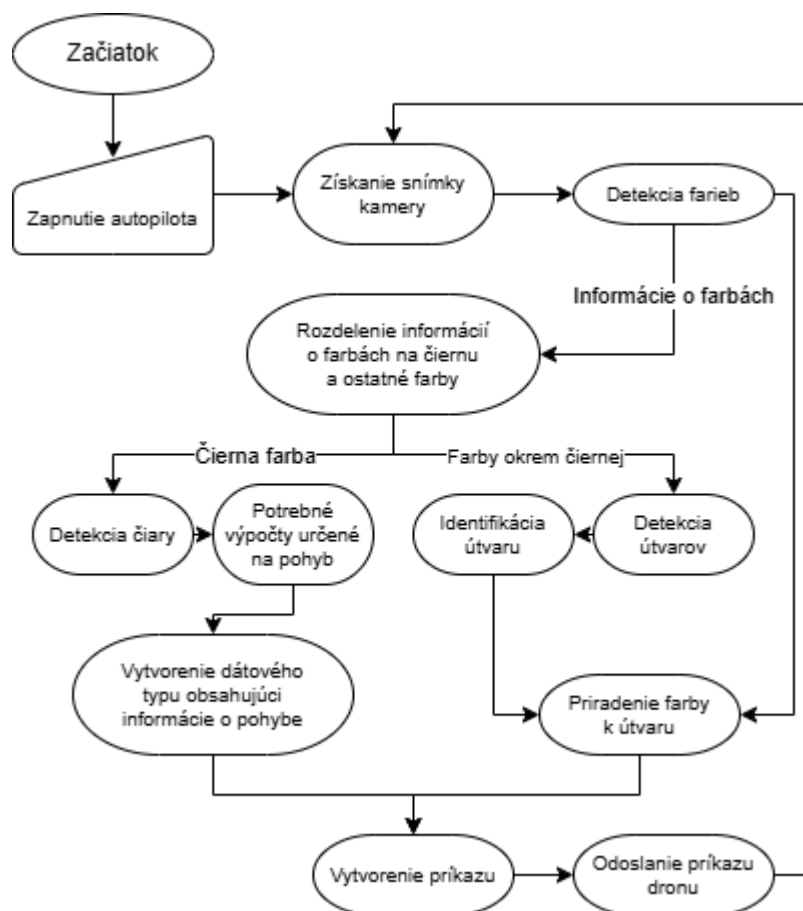
od zapnutia autopilota až po odosielanie príkazov dronu na základe analýzy vizuálnych informácií z obrazu kamery dronu. Kľúčovým prvkom je spracovanie snímky kamery, detekcia farieb a následná identifikácia objektov v prostredí. Tieto dáta sa potom využívajú na výpočty potrebné pre pohyb drona a následné plnenie úloh. Diagram zároveň zobrazuje spôsob, akým sú vizuálne informácie kategorizované a spracované, čo umožňuje efektívne a prehľadné riadenie drona.

Dron postupne vykonáva jednotlivé kroky znázornené na diagrame počas autonómneho letu. Po zapnutí autopilota sa začne cyklus, ktorý na začiatku získava snímku z kamery. Táto snímka je následne spracovaná softvérom, ktorý deteguje farby a rozdeľuje ich do dvoch hlavných kategórií. Čierna farba, ktorá sa využíva na navigáciu podľa čiernej čiary a ostatné farby, ktoré reprezentujú rôzne útvary v prostredí.

Ak softvér identifikuje čiernu farbu, spustí sa proces detekcie čiary. Pri tomto procese sa vykonávajú potrebné výpočty, ako napríklad výpočet odchýlky od stredu čiary a uhol, do ktorého sa čiara stáča. Výpočty nám tak zabezpečia správny pohyb drona podľa preddefinovaného značenia. Ak sa v snímke obrazu nachádzajú aj iné farby, softvér sa zameria na identifikáciu útvarov a ich analýzu. Po úspešnej detekcii sa jednotlivým útvarom priradí konkrétna farba, pod ktorou boli nájdené.

Na základe tohto spracovania vizuálnych informácií softvér generuje vlastnú dátovú štruktúru obsahujúcu všetky potrebné informácie o ďalších krokoch drona. Táto dátová štruktúra slúži ako podklad pre vytvorenie konkrétnych príkazov, ktoré bližšie určujú smer letu a reakciu na prekážky. Ako finálny krok softvér odosiela príkazy dronu, ktorý ich okamžite vykonáva a prispôbuje svoj pohyb aktuálnym podmienkam v prostredí.

Vývojový diagram autonómneho letu nám slúži ako kľúčový nástroj na vizualizáciu procesov, ktoré riadia dron. Jasne nám ukazuje ako jednotlivé kroky a moduly spolupracujú na zabezpečení efektívneho a spoľahlivého autonómneho letu. Jeho vytvorenie nám pomohlo lepšie pochopiť vzťahy medzi rôznymi časťami softvéru a identifikovať kritické body, ktoré bolo potrebné neskôr optimalizovať. Diagram nám zároveň poslúžil ako dôležitý podklad pri implementácii logiky autonómneho letu.



Obrázok 9 - Vývojový diagram autonómneho letu

5 Počítačové videnie

Počítačové videnie patrí medzi kľúčové moduly nášho softvéru, pretože umožňuje dronu spracovávať obrazové dáta, napríklad identifikovať farebné útvary a na základe nich vykonávať autonómne rozhodnutia. Táto technológia napodobňuje schopnosť ľudského oka a mozgu vnímať, rozpoznávať a interpretovať vizuálne informácie, čo je nevyhnutné pre presnú navigáciu a vykonávanie úloh v dynamickom prostredí, v ktorom sa dron pohybuje.

V rámci tohto projektu je hlavnou úlohou počítačového videnia umožniť dronu identifikovať vizuálne prvky, ako sú farby a útvary pod ním. Počítačové videnie sa konkrétne zameriava na detekciu farieb a rozpoznávanie útvarov. Detekcia farieb je prvým krokom spracovania obrazových dát, ktorý umožňuje identifikovať objekty na základe ich farby. Tento proces je možný vďaka využitiu kamery drona, ktorá neustále sníma okolie a poskytuje nám potrebné dáta na analýzu. Po detekcii

farieb sa vykonáva ďalší krok, a to rozpoznávanie útvarov. To sa vykonáva na všetkých farbách okrem čiernej, keďže tá sa využíva neskôr pri sledovaní čiernej čiary.

Tieto techniky spolu zabezpečujú presné vnímanie prostredia, čo je nevyhnutné pre efektívnu realizáciu autonómnych krokov. Kombinácia detekcie farieb a rozpoznávania útvarov tvorí základ pre ďalšie kroky v navigácii a rozhodovaní.

5.1 Detekcia farieb

Detekcia farieb zohráva kľúčovú úlohu pri spracovaní obrazu, pretože umožňuje identifikovať a segmentovať jednotlivé objekty podľa ich farby. Tento proces je jedným z prvých krokov pri analýze obrazu, ktorý sa následne kombinuje s ďalšími, ako je rozpoznávanie útvarov. Na detekciu farieb sa využíva nami vytvorená trieda *ColorRecognition*, ktorá z obrazu extrahuje farebné oblasti a spracováva ich pre ďalšie použitie.

Obraz zachytený kamerou drona je pôvodne v RGB (Red-Green-Blue) farebnom modeli, ktorý sa bežne používa na zobrazovanie farieb na obrazovkách. Avšak na presnejšiu detekciu farieb sa obraz konvertuje do modelu HSV (Hue-Saturation-Value).

Prečo práve model HSV a nie HSL (Hue-Saturation-Lightness)? Rozdiel spočíva v tom, že v modeli HSL dosahuje farba s maximálnou svetlosťou čistú bielu farbu, zatiaľ čo v HSV je farba s maximálnym jasom stále farebná, ale výraznejšia. Skvelým príkladom je farebný objekt nasvietený bielym svetlom. Keď na červený objekt svietime jasným bielym svetlom, objekt stále vyzerá červeno, ale jeho jasnosť sa zvýši. Tento efekt je v modeli HSV prirodzenejší, pretože jas (value) tu reprezentuje intenzitu farby, zatiaľ čo v HSL svetlosť (lightness) môže zmeniť samotnú farbu a môže viesť k tomu, že farba pri maximálnej svetlosti získa bielu.

Model HSV je vhodnejší, pretože oddeľuje informácie o odtieni farby (hue), jej sýtosť (saturation) a jas (value). Vďaka tomu umožňuje lepšiu segmentáciu farieb aj pri rôznych svetelných podmienkach.

5.1.1 Inicializácia triedy

Inicializácia triedy *ColorRecognition* zabezpečuje načítanie farebných rozsahov z JSON súboru, ktorý obsahuje informácie o jednotlivých farbách. Tento súbor obsahuje pole objektov, pričom každý objekt reprezentuje jednu farbu a definuje jej názov spolu s príslušnými hranicami v HSV farebnom modeli. Trieda zároveň načítava konfiguračné hodnoty z INI súboru, kde sú uložené ďalšie parametre potrebné na detekciu farieb, ako napríklad minimálna plocha kontúry alebo relatívna cesta k JSON súboru.

Na spracovanie dát slúži metóda *load_color_bounds*, ktorá načíta obsah JSON súboru a extrahuje z neho potrebné informácie o farbách. Pre každú farbu uloží jej názov do poľa *color_names*, a dvojicu hodnôt predstavujúcich hornú a dolnú hranicu rozsahu do poľa *color_bounds*. Aby bolo možné s týmito hodnotami efektívne manipulovať pri spracovaní obrazu, sú konvertované do polí knižnice NumPy, čím sa zabezpečí rýchlejšia manipulácia s dátami.

5.1.2 Rozpoznávanie jednotlivých farieb

Metóda *detect_colors* je zodpovedná za identifikáciu farieb v danej snímke obrazu. Vstupným parametrom tejto metódy je snímka vo formáte N-dimenzionálneho poľa, z ktorej sa najskôr vytvorí duplikát, aby sa zachovala pôvodná snímka. Následne sa konvertuje z BGR farebného modelu do HSV farebného modelu. HSV sme vybrali, pretože lepšie reprezentuje farebné odtiene, sýtosť a jas. Tento model nám teda umožňuje lepšie a presnejšie rozpoznať jednotlivé farby.

Metóda následne prechádza cez zoznam preddefinovaných rozsahov a názvov farieb. Pre každý rozsah farby sa vytvorí binárna maska pomocou metódy *apply_mask*, ktorá identifikuje jednotlivé pixely v obraze spadajúce do daného rozsahu farby. Po aplikovaní masky využijeme funkciu *findContours* z knižnice OpenCV na vyhľadanie kontúr vo výslednej maske, čím sa identifikujú časti obrazu, ktoré obsahujú rozpoznanú farbu. Ak sú v maske nájdené kontúry, metóda vyberie najväčšiu kontúru.

Ak veľkosť najväčšej nájdenej kontúry presiahne minimálnu prahovú hodnotu načítanú z konfiguračného súboru, považujeme ju za relevantnú. Na zvýraznenie polohy detegovanej farby sa použije funkcia *boundingRect* z knižnice OpenCV, ktorá nám vráti

súradnice a rozmery obdĺžnika obopínajúceho túto kontúru. Informácie o detegovanej farbe vrátane jej názvu a pozícii v obraze sa uložia do zoznamu *detected_colors* vo forme slovníka. Tento zoznam je na konci metódy vrátený ako výstup metódy *detect_colors*.

5.1.3 Odstránenie šumu v maske

Po vytvorení binárnej masky pre detegované farby je dôležité odstrániť šum, ktorý môže vzniknúť z rôznych dôvodov, ako sú nedokonalosti vstupného obrazu zo snímania dronom alebo drobné odchýlky vo farebnom rozsahu pri detekcii farieb, napríklad v dôsledku zmeny osvetlenia. Šum sa môže prejavovať ako malé biele bodky v častiach obrazu, kde sa nenachádza žiadny relevantný objekt, alebo ako prerušené oblasti v rámci väčších a relevantných objektov, ktoré neboli správne označené ako súčasť masky.

Na odstránenie tohto šumu z masky využíva metóda *remove_noise* morfologické operácie. Morfologické operácie sú také operácie, ktoré sa zameriavajú na štruktúru a formu objektov v obraze, namiesto samotných farieb či intenzitu pixelov. Tieto operácie využívajú v našom prípade jadro, čo je zvyčajne malá matica, ktorá sa posúva po obraze. Týmto spôsobom sa mení štruktúra obrazu, ako je rozširovanie alebo zužovanie objektov.

Metóda *remove_noise* využíva operáciu morfologického otvorenia, známu pod názvom *MORPH_OPEN* v knižnici OpenCV, ktorá pracuje s malou maticou nazývanou jadro. V našom prípade má jadro rozmery päť na päť prvkov. Tento typ operácie je kombináciou erózie, ktorá spôsobuje zúženie svetlých oblastí a dilatácie, ktorá naopak zväčšuje svetlé oblasti. Erózia nám najprv odstráni malé biele bodky, zatiaľ čo dilatácia nám zabezpečí, že väčšie objekty zostanú neporušené a nezmenšia sa.

5.2 Rozpoznávanie útvarov

Rozpoznávanie útvarov je ďalším dôležitým krokom pri analýze obrazu, ktorý nadväzuje na detekciu farieb a umožňuje nám identifikovať objekty na základe ich tvaru. Tento proces zohráva kľúčovú úlohu pri plnení autonómnych úloh, kde je potrebné rozlíšiť rôzne geometrické útvary. Na tento účel využívame nami vytvorenú triedu *ShapeRecognition*, ktorá nepracuje priamo s pôvodným obrazom, ale analyzuje

binárnu masku vytvorenú kombináciou viacerých farebných masiek vygenerovaných pri detekcii farieb. Táto maska obsahuje segmentované oblasti objektov, pričom trieda identifikuje ich kontúry a na základe počtu vrcholov a rozmerov určuje konkrétny typ geometrického útvaru. Výsledkom je zoznam rozpoznaných útvarov, ktoré sú následne spojené s príslušnou farbou.

5.2.1 Inicializácia triedy

Na správne fungovanie rozpoznávania útvarov je potrebné, aby trieda *ShapeRecognition* mala prístup k dôležitým parametrom, ktoré ovplyvňujú presnosť a stabilitu detekcie. Pri vytvorení inštancie triedy sa najskôr načítajú hodnoty z konfiguračného súboru. Trieda zároveň uchováva zoznam sledovaných útvarov, ktorý umožňuje efektívne spracovanie a zabraňuje opakovanej detekcii rovnakých objektov v krátkom časovom úseku. Táto inicializácia tvorí základ pre všetky ďalšie operácie spojené s rozpoznávaním a sledovaním útvarov v obraze.

5.2.2 Detekcia útvarov

Hlavnou úlohou rozpoznávania útvarov je detekcia a identifikácia objektov nachádzajúce sa v maske, ktorá bola vytvorená počas detekcie farieb. Trieda *ShapeRecognition* na tento účel využíva metódu *detect_shapes*, ktorá analyzuje obrisy objektov a určuje ich tvar na základe geometrických vlastností.

Najprv sa z poskytnutej masky extrahujú kontúry, ktoré predstavujú hranice jednotlivých objektov. Ak sa v obraze nenachádzajú žiadne kontúry, zoznam sledovaných útvarov sa vyprázdni, čím sa zabráni nesprávnemu ukladaniu neaktuálnych útvarov. Následne sa pre každý nájdený obrys vytvorí štruktúra, respektíve slovník obsahujúci názov rozpoznaného útvaru, jeho polohu v obraze a počítadlo výskytov.

Aby sa predišlo duplicitnému spracovaniu tých istých objektov v po sebe idúcich snímkach, algoritmus overuje, či sa daný útvar už nachádza v zozname sledovaných útvarov. Ak áno, jeho výskyt sa zaznamená a zvýši sa jeho počítadlo. Ak ide o nový útvar, pridá sa do zoznamu na sledovanie. Tento postup zaisťuje stabilnú detekciu a minimalizuje falošné pozitíva spôsobené dočasnými zmenami v obraze.

Nakoniec sa z celkového zoznamu vyberú len tie útvary, ktoré dosiahli stabilnú detekciu, čo znamená, že boli potvrdené dostatočným počtom po sebe idúcich snímok.

Ostatné objekty zostávajú v sledovaní, až kým nesplnia podmienku stability alebo nezmiznú z obrazu.

5.2.3 Identifikácia útvarov

Na správne rozpoznanie tvaru objektu je potrebné analyzovať jeho kontúry a určiť počet jeho vrcholov. Trieda *ShapeRecognition* využíva algoritmus aproximácie kontúr, ktorý zjednodušuje tvar objektu na základe jeho dĺžky obvodu. Týmto spôsobom získame súbor bodov, ktoré reprezentujú hlavné vrcholy útvaru. Na základe ich počtu môžeme rozlíšiť základné geometrické útvary.

Pri štvoruholníkoch je navyše potrebné zistiť pomer strán, aby sme mohli odlíšiť štvorce od obdĺžnikov. Ak má tvar viac ako štyri vrcholy, považujeme ho za kruh. Tento prístup nám umožňuje efektívne a spoľahlivo klasifikovať objekty v obraze a následne ich využiť pri autonómnych úlohách.

5.2.4 Sledovanie a stabilizácia detegovaných útvarov

Keď už máme z obrázka extrahované jednotlivé útvary, nestačí ich len jednorazovo identifikovať. V dynamickom prostredí, kde sa dron pohybuje a obraz sa neustále mení, sa môže stať, že sa rovnaký objekt deteguje nesprávne alebo dočasne zmizne z obrazu v dôsledku šumu. Preto bolo dôležité vymyslieť algoritmus, ktorý nám zabezpečí, že rozpoznané útvary sú konzistentné a nie sú mylne interpretované ako iné alebo nové objekty.

Na tento účel využívame metódu *is_close*, ktorá nám umožňuje určiť, či sú dva útvary dostatočne blízko na to, aby boli považované za ten istý objekt. Táto kontrola prebieha na základe vzdialenosti stredov obdĺžnikov, ktoré ohraničujú detegované útvary. Ak je vzdialenosť medzi dvoma útvarmi menšia alebo rovná definovanému prahovému limitu, považujeme ich za rovnaké. Tento prah je konfigurovateľný a umožňuje nám prispôsobiť citlivosť sledovania podľa potrieb konkrétnej aplikácie.

Aby sme sa vyhli opakovanému pridávaniu už rozpoznaných útvarov, využívame ďalšiu metódu *is_tracked*, ktorá kontroluje, či už daný útvar evidujeme v zozname sledovaných objektov. Sledované útvary sú porovnávané nielen na základe názvu, ale aj ich polohy v obraze. Ak sa nový útvar nachádza v blízkosti niektorého z už

sledovaných a zároveň má rovnaký tvar, považujeme ho za existujúci objekt a nevytvárame nový záznam.

Tento proces sledovania je nevyhnutný na to, aby sme zabezpečili spoľahlivú detekciu a minimalizovali chyby spôsobené dočasnými odchýlkami pri spracovaní obrazu. Vďaka nemu vieme lepšie interpretovať snímky a správne priradiť identifikované útvary k ich skutočným fyzickým objektom.

5.2.5 Aktualizácia sledovaných útvarov

Aby bola detekcia útvarov spoľahlivá, vytvorili sme si pomocné metódy, ktoré v reálnom čase sledujú ich výskyt. Metóda *increase_counter* zvyšuje počítadlo útvarov, ktoré sa v obraze objavujú opakovane a sú rovnaké. Ak útvar dosiahne nastavený prah stability, považujeme ho za správne detegovaný a zaradíme ho medzi rozpoznané útvary, ktoré sú vrátené na konci hlavnej metódy. Naopak, ak sa prestane objavovať, zo sledovania ho odstránime.

Nové útvary, ktoré nie sú blízko existujúcim, pridáva do sledovania metóda *track_new_shape*. Tento prístup zabezpečuje, že sa detekcia prispôbuje zmenám v obraze a eliminuje falošné identifikácie spôsobené šumom alebo krátkodobými chybami.

6 Pohyb drona

Autonómny pohyb drona a vykonávanie úloh je druhým kľúčovým modulom nášho softvéru. Pohyb drona je riadený na základe sledovania čiernej čiary, ktorá slúži ako navigačný prvok. Tento proces spočíva v spracovaní obrazu z kamery drona, identifikácii čiary a následnom výpočte jej uhla a polohy voči stredu obrazu. Hlavným cieľom je udržať dron čo najpresnejšie na trase, pričom sa zohľadňuje jeho vzdialenosť od stredu a uhol, do ktorého sa čiara stáča.

Pri detekcii čiary sa využívajú algoritmy spracovania obrazu, ktoré hľadajú najvýraznejšiu kontúru na vopred spracovanom čiernobielym obraze. Po jej identifikácii sa vypočíta jej stred a uhol natočenia. Na základe týchto údajov sa vypočíta korekcia pohybu, aby dron ostal v správnej trajektórii.

Riadenie pohybu je realizované prostredníctvom odosielenia príkazov dronu pomocou triedy *Drone*. Každý príkaz obsahuje informácie o posune v jednotlivých osiach a otočení. Systém zabezpečuje plynulé manévrovanie tak, aby dron neprekročil maximálne dovolené odchýlky a dokázal reagovať na zmeny trasy.

Okrem sledovania čiary sa využívajú aj ďalšie nami vytvorené dátové štruktúry na riadenie správania drona. Jednou z nich je štruktúra *Shape*, ktorá slúži na uloženie informácií o rozpoznanom útvere. Táto dátová štruktúra sa ďalej využíva na vykonávanie autonómnych úloh. Umožňuje dronu reagovať nielen na trasu, ale aj na objekty v prostredí.

Celý proces je navrhnutý tak, aby bol autonómny a spoľahlivý aj pri menších odchýlkach či prerušení línie v obraze. Ak dôjde k strate obrazového vstupu, systém je schopný pokúsiť sa o opätovnú lokalizáciu čiary na základe predchádzajúcich údajov a predpokladaného pohybu drona.

6.1 Sledovanie čiary

Sledovanie čiary je kľúčová úloha pre autonómne riadenie dronu, ktorá umožňuje sledovanie vopred určenej trasy. Implementácia využíva počítačové videnie na spracovanie obrazu na detekciu čiary a výpočet potrebných korekcií pohybu dronu. Na spracovanie obrazu sa používa OpenCV, kde sa z kamery dronu získava obraz a následne sa filtrujú pixely podľa farby sledovanej čiary. Po konvertovaní obrazu do binárnej masky sa pomocou kontúr zisťuje poloha a orientácia čiary.

Trieda *Line* zabezpečuje detekciu čiary v zadanej maske obrazu, výpočet súradníc stredu čiary, určenie uhla natočenia čiary a vizualizáciu údajov priamo v obraze. Po spracovaní obrazu sa vypočíta odchýlka dronu voči stredu čiary, ktorý sa následne prevedie na reálne súradnice v centimetroch. Uhol čiary sa tiež berie do úvahy na korekciu smeru pohybu. Na základe týchto údajov sa vytvára dátová štruktúra *Movement*, ktorá sa následne posiela do dátovej štruktúry *Command* na vytvorenie príkazu pre dron. Tento systém umožňuje dronu automaticky korigovať svoju pozíciu a orientáciu, čím sa zabezpečuje plynulé sledovanie čiary.

6.2 Riadenie pohybu drona

Riadenie pohybu drona je založené na korekciách v reálnom čase, ktoré zohľadňujú rôzne faktory, ako sú odchýlka drona od stredu čiary, uhol a rýchlosť pohybu. Systém riadenia pohybu pracuje s príkazmi, ktoré sú odosielané dronu, pričom tieto príkazy obsahujú informácie o požiadavkách na pohyb v jednotlivých osiach (x, y, z) a uhol (yaw).

Tieto informácie sú ukladané do dátovej štruktúry *Movement*, ktorá je posielaná ako typ príkazu pre pohyb do metódy *send_command* v triede *Drone*. Pohyb dronu je takto optimalizovaný a umožňuje plynulú navigáciu v reálnom čase, pričom stále zohľadňuje aktuálnu pozíciu dronu a požiadavky na jeho správne nasmerovanie.

6.3 Plnenie úloh podľa farebných útvarov

Plnenie úloh podľa farebných útvarov je tiež jedným z kľúčových prvkov riadenia drona v rámci tohto projektu. Príkazy pre pohyb drona sú vytvárané na základe analýzy farebných útvarov detegovaných počítačovým videním. Každý farebný útvar je identifikovaný a jeho vlastnosti sú uložené do dátovej štruktúry *Shape*. Táto štruktúra obsahuje informácie o tvare, farbe, pozícii a rozmeroch objektu, čo umožňuje presnú analýzu a spracovanie daného tvaru.

Na základe týchto informácií sa generuje príkaz typu *action*, ktorý určuje, akú konkrétnu akciu má dron vykonať. Tieto príkazy môžu zahŕňať pohyb drona v rôznych smeroch, zmenu orientácie (uhla), alebo iné špecifické činnosti potrebné na splnenie úlohy. Príkazy sú následne odoslané do metódy *send_command* triedy *Drone*, ktorá zabezpečí vykonanie požiadavky.

Tento systém umožňuje dronu flexibilne reagovať na rôzne objekty v jeho okolí a vykonávať úlohy, ktoré sú prispôbené detegovaným farebným tvarom. Výsledkom je efektívna a adaptívna navigácia, ktorá využíva vizuálne informácie na presné vykonávanie požiadaviek na pohyb a akcie.

7 Grafické rozhranie programu

Po tom ako sme úspešne naprogramovali dve kľúčové časti programu, počítačové videnie a pohyb drona, tak ich musíme prepojiť, aby medzi sebou vedeli komunikovať a aby používateľ videl všetky potrebné informácie na obrazovke. Na tento účel sme navrhli moderné grafické rozhranie, pričom prvá verzia GUI bola vytvorená pomocou knižnice Tkinter ako dočasné riešenie. Toto riešenie nám umožnilo rýchlo implementovať základnú funkcionality a overiť komunikáciu medzi systémami. Po prvotných testoch sme však prešli na knižnicu PySide6 s technológiou Qt Quick a vyvíjali GUI pomocou QML. Ponúkli nám tak oveľa flexibilnejšie možnosti pre moderné a vizuálne prepracované rozhranie.

Samotné GUI je rozdelené na dve hlavné časti, a to je hlavná aplikácia a okno s kamerovým výstupom. Tento prístup umožňuje lepšiu organizáciu prvkov a zároveň zvyšuje prehľadnosť pri práci s aplikáciou. Hlavné rozhranie poskytuje informácie o stave drona, pričom obsahuje indikátory ako úroveň batérie, aktuálnu výšku, teplotu a stav pripojenia. Nechýba ani panel na ovládanie drona vrátane tlačidla na zapnutie autonómneho letu. Používateľ tak môže ovládať dron bez potreby priamych príkazov cez konzolu.

Dôležitou súčasťou je aj okno s kamerovým výstupom, ktorý umožňuje sledovanie diania v reálnom čase. Používateľ má možnosť prepínať medzi rôznymi režimami zobrazenia, ako je čistý obraz priamo z kamery alebo zobrazenie spracovania obrazu.

Aby aplikácia bola vizuálne príťažlivá a zároveň funkčná, použili sme moderný dizajn s minimalistickými prvkami a hladkým prechodom medzi jednotlivými stavmi aplikácie. Kombinácia svetlých a tmavých prvkov zaisťuje dobrú čitateľnosť pri rôznych svetelných podmienkach. Navyše, aplikácia automaticky nastavuje ikony na základe systému používateľa, čím sa prispôsobuje svetlému alebo tmavému režimu. V aplikácii je dostupná aj sekcia nastavení, kde môže používateľ upravovať parametre drona a aplikácie podľa svojich potrieb.

Celý systém je navrhnutý tak, aby bola komunikácia medzi backendom, logikou softvéru, a GUI efektívna. Na tento účel sa využívajú Qt signály a sloty, ktoré

zabezpečujú okamžitú odozvu rozhrania na zmeny stavu drona. Napríklad, ak sa zmení výška drona, príslušný signál sa automaticky vyvolá a aktualizuje sa zobrazená hodnota v GUI. Týmto spôsobom je možné plynule synchronizovať všetky relevantné dáta bez nutnosti neustáleho dotazovania.

Celkovo tak grafické rozhranie poskytuje prehľadný, responzívny a moderný spôsob ovládania drona, pričom používateľ má k dispozícii všetky potrebné informácie a nástroje na efektívnu prácu. Dôležitým prvkom je aj modularita riešenia, ktorá umožňuje v budúcnosti jednoducho rozširovať funkcionality aplikácie.

Záver

V tejto maturitnej práci sme sa zamerali na vývoj autonómneho drona s využitím počítačového videnia, ktorý je schopný detegovať farby, rozpoznávať útvary a vykonávať autonómne rozhodnutia v dynamickom prostredí. Cieľom projektu bolo nielen vyvinúť technológie pre autonómny pohyb drona, ale aj implementovať pokročilé algoritmy pre spracovanie obrazových dát, ktoré dronu umožňujú identifikovať útvary a plniť úlohy bez ľudského zásahu.

V priebehu realizácie projektu sme využili rôzne technológie a nástroje, ktoré boli kľúčové pre jeho úspech a dostatočne nám zľahčili vývoj. Medzi nimi bol aj samotný dron DJI Tello, ktorý slúžil ako základ pre náš autonómny systém a programovací jazyk Python, ktorý sme použili na vývoj celého softvéru. Kľúčovou súčasťou softvéru boli aj knižnice OpenCV a NumPy, ktoré nám umožnili implementovať funkcionality počítačového videnia, ako aj knižnica DJITelloPy, ktorá nám poskytla rozhranie pre ovládanie drona.

Jednou z hlavných častí projektu bola implementácia počítačového videnia, ktoré nám umožnilo detekciu farieb a rozpoznávanie útvarov. Tieto technológie sú nevyhnutné pre identifikáciu objektov a správne vykonávanie autonómnych rozhodnutí. Okrem toho sme použili aj ďalšie nástroje, ako je Tkinter a PySide6 s Qt frameworkom na vytvorenie používateľského rozhrania pre jednoduché ovládanie softvéru.

V prvých krokoch praktickej realizácie sme sa zamerali na návrh a koncept softvéru, kde sme vytvorili vývojový diagram autonómneho letu a podrobne sa venovali aj konštrukcii a tlači súčiastok. Počas vývoja sme sa stretli s rôznymi výzvami, najmä pri optimalizácii algoritmov spracovania obrazu a integrovaní rôznych technológií do jedného funkčného systému. Tieto výzvy sme úspešne prekonali a výsledkom je funkčný dron, ktorý dokáže detegovať farby, rozpoznávať útvary, navigovať sa pomocou čiernej čiary a autonómne vykonávať všetky úlohy.

Na základe dosiahnutých výsledkov môžeme skonštatovať, že projekt sa podarilo úspešne dokončiť a splnil stanovené ciele. Tento projekt nám ukázal potenciál kombinácie rôznych technológií a ich schopnosť vytvárať inteligentné zariadenia schopné vykonávať komplexné úlohy bez ľudského zásahu.

ZOZNAM BIBLIOGRAFICKÝCH ODKAZOV

- [1] IBM. What is Automation?. [online]. [navštívené 10.03.2025]. Dostupné na: <https://www.ibm.com/think/topics/automation>
- [2] BlackBerry QNX. Ultimate Guide to Autonomous Systems. [online]. [navštívené 10.03.2025]. Dostupné na: <https://blackberry.qnx.com/en/ultimate-guides/autonomous-systems>
- [3] Vation Ventures. Autonomous Systems: Definition, Explanation, and Use Cases. [online]. [navštívené 10.03.2025]. Dostupné na: <https://www.vationventures.com/glossary/autonomous-systems-definition-explanation-and-use-cases>
- [4] Built In. What is a Drone?. [online]. [navštívené 10.03.2025]. Dostupné na: <https://builtin.com/drones>
- [5] Ryze Tech. Tello. [online]. [navštívené 10.02.2025]. Dostupné na: <https://www.ryzerobotics.com/tello>
- [6] Integrating Computing Across the Curriculum. Introduction to Scratch. [online]. [navštívené 10.03.2025]. Dostupné na: https://www.uab.edu/icac/images/Scratch_Guides/Intro_to_Scratch.pdf
- [7] Ryze Tech. Tello SDK 2.0 User Guide. [online]. [navštívené 10.03.2025]. Dostupné na: <https://dl-cdn.ryzerobotics.com/downloads/Tello/Tello%20SDK%202.0%20User%20Guide.pdf>
- [8] TECHA, Mark. Understanding Common Ports Used in Networks for TCP and UDP Usage. [online]. 11.10.2024. [navštívené 10.03.2025]. Dostupné na: <https://blog.netwrix.com/common-ports>
- [9] Python. What is Python? Executive Summary. [online]. [navštívené 10.03.2025]. Dostupné na: <https://www.python.org/doc/essays/blurb/>
- [10] Python. Comparing Python to Other Languages. [online]. [navštívené 10.03.2025]. Dostupné na: <https://www.python.org/doc/essays/comparisons/>

- [11] FUENTES ESCOTÉ, Damià. DJITelloPy. [online]. [navštívené 10.03.2025]. Dostupné na: <https://github.com/damiafuentes/DJITelloPy/blob/master/README.md>
- [12] Open Source Computer Vision Library. Introduction. [online]. [navštívené 10.03.2025]. Dostupné na: <https://docs.opencv.org/4.x/d1/dfb/intro.html>
- [13] NumPy. NumPy. [online]. [navštívené 25.03.2025]. Dostupné na: <https://numpy.org/>
- [14] Python. Tkinter – Python interface to Tcl/Tk. [online]. [navštívené 10.03.2025]. Dostupné na: <https://docs.python.org/3/library/tkinter.html>
- [15] The Qt Company Ltd.. Qt for Python. [online]. [navštívené 10.03.2025]. Dostupné na: <https://doc.qt.io/qtforpython-6/>
- [16] The Qt Company Ltd.. Getting Started. [online]. [navštívené 10.03.2025]. Dostupné na: <https://doc.qt.io/qtforpython-6/gettingstarted.html>
- [17] Strikwerda en Dehue. What is 3D printing?. [online]. [navštívené 26.03.2025]. Dostupné na: <https://3dprinting.com/what-is-3d-printing/>
- [18] Blender. Features. [online]. [navštívené 25.03.2025]. Dostupné na: <https://www.blender.org/features/>